UT 5. Programación web dinámica

Índice de contenidos

1. Mecanismos de separación de la lógica de negocio....2
1.1. Patrón MVC....2
1.2. Desarrollo con frameworks....3
2. Symfony....3
2.1. Instalación y configuración....3
2.2. Estructura de un proyecto Symfony....5
2.3. Creando la primera página....5
2.3.1. Ruta y controlador....5
2.3.2. Mostrando una plantilla....6
2.4. Rutas....6
2.5. Controladores....7
2.5.1. Obtener URLs....7
2.5.2. Redirección....8
2.5.3. Accediendo a datos de la petición (GET, POST, Cookies…)....8
2.6. Creando y usando plantillas Twig....8
2.6.1. Plantillas....9
2.6.2. Herencias en plantillas....10
2.6.3. Incluyendo unas plantillas dentro de otras....11
2.6.4. Enlaces en las plantillas....12
2.6.5. Enlazando Assets....13
2.6.6. Incluyendo JavaScript y CSS en twig....13
2.7. Formularios....14
2.7.1. Creando una entidad....14
2.7.2. Construyendo el formulario....14
2.7.3. Mostrando el formulario....16
2.7.4. Gestionando el formulario....16
2.8. Base de datos y Doctrine....17
2.8.1. Instalando doctrine....17
2.8.2. Configurando la base de datos....17
2.8.3. Creando la entidad....18
2.8.4. Persistiendo los objetos en la base de datos....20
2.8.5. Obteniendo objetos de la base de datos....21
2.8.6. Editando objetos....23
2.8.7. Borrando objetos....24

1. Mecanismos de separación de la lógica de negocio

En informática y ciencias de la computación, el término lógica de negocio (Business Logic) se refiere a aquella parte de un sistema que se encarga de manejar los datos para generar una respuesta a las peticiones del cliente.

Un sitio web típico tiene tres tipos de funcionalidad:

1. El código que implementa la l**ógica del negocio**,
2. El código que implementa la **interacción con el usuario** y
3. El código que imprime páginas HTML (**presentación**).

Los programadores usualmente mezclaban estos tres tipos de código en un solo gran archivo PHP que lo hace doloroso de probar y mantener, especialmente cuando escribimos un sitio web grande.

La ingeniería de software ha evolucionado mucho en este sentido, elaborando principios y patrones de desarrollo que nos permitan elaborar un código más estructurado (mejor organizado), modular (componentes independientes intercambiables) y fiable (con menos errores). Uno de esos patrones es el conocido como patrón de diseño **MVC: "Modelo - Vista - Controlador"**.

1.1. Patrón MVC

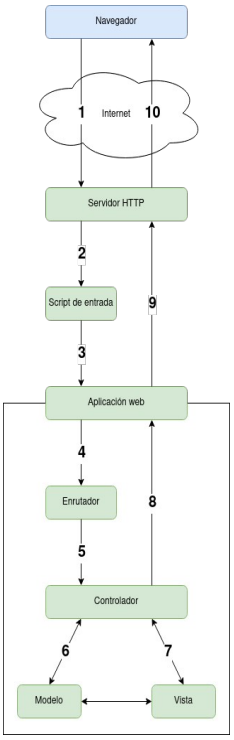
Existen varios métodos que permiten separar la lógica de presentación (en nuestro caso, la que genera las etiquetas HTML) de la lógica de negocio, donde se implementa la lógica propia de cada aplicación. El patrón MVC pretende dividir el código en tres partes, dedicando cada una a una función definida y diferenciada de las otras.

- **Modelo.** Es el encargado de manejar los **datos propios de la aplicación** y del problema que trata. Debe proveer mecanismos para obtener y modificar la información del mismo, **normalmente utilizando clases**. Si la aplicación utiliza algún tipo de almacenamiento para su información (como un SGBD), tendrá que encargarse de almacenarla y recuperarla.
- **Vista.** Es la parte del modelo que se encarga de la **interacción con el usuario**. En esta parte se encuentra el código necesario para generar el interfaz de usuario (en nuestro caso en HTML), según la información obtenida del modelo.
- **Controlador.** En este módulo se decide **qué se ha de hacer**, en función de las acciones del usuario con su interfaz. Con esta información, interactúa con el **modelo** para indicarle las acciones a realizar y, según el resultado obtenido, envía a la **vista** las instrucciones necesarias para generar el nuevo interfaz.

La gran ventaja de este patrón de programación es que genera código muy estructurado, fácil de comprender y de mantener.

Para saber más ...

*En la web puedes encontrar algunos ejemplos de implementación del modelo MVC en PHP. Échale un vistazo al siguiente **artículo**.*



Para entender mejor como funciona esto recordemos que cualquier sitio web solo es un script PHP que recibe una **petición HTTP desde el servidor web y produce una repuesta HTTP**. A continuación se muestra como una petición HTTP es procesada por una aplicación MVC y como se genera la respuesta:

Primero un visitante del sitio web ingresa la URL en su navegador web, y el navegador web envía la petición al servidor web.

1. El servidor web recibe la petición y ejecuta el script de entrada.
2. El script de entrada lo único que hace es crear una instancia de la clase principal de la aplicación.
3. La aplicación llama al componente **enrutador** y le indica la ruta de la petición.
4. El enrutador analiza la ruta y determina a que **controlador** pasar la petición. Si coincide una ruta el controlador es instanciado y se llama al **método de acción** apropiado.

5. Para procesar los datos entrantes el controlador **instancia las clases de modelo** apropiadas y **llama a sus métodos**. Las clases de modelo usa los algoritmos de la lógica de negocio para procesar los datos de entrada y regresar los datos de salida. Los algoritmos de la lógica de negocios son específicos de nuestra aplicación y típicamente incluye recuperar datos de la base de datos, manejar archivos, interactuar con sistemas externos, etc.

6. Lo que resulta de llamar a los modelos se pasa al **script de vista** correspondiente para generar el código HTML. El script de vista usa los datos que provee el modelo para imprimir la página HTML.

7. El controlador pasa la respuesta HTTP resultante a la aplicación.

8. La aplicación devuelve la respuesta al servidor.

9. El servidor devuelve la respuesta al navegador y este muestra la página al usuario.

Ahora tenemos una idea de como los modelos, las vistas y los controladores cooperan para generar una salida HTML.

1.2. Desarrollo con frameworks

Aunque puedes programar utilizando MVC por tu cuenta, es más habitual utilizar el patrón MVC en conjunción con un **framework** o marco de desarrollo. Existen numerosos frameworks disponibles en PHP, muchos de los cuales incluyen soporte para MVC.

Un **framework** es una estructura conceptual y tecnológica de soporte definido, normalmente con **artefactos** o módulos de software concretos, que puede servir de base para la organización y desarrollo de software.

En la actualidad se ha popularizado el uso de frameworks para desarrollo. Entre otras **ventajas** de usarlos podemos destacar:

- **Evitar escribir código repetitivo**. La mayoría de los proyectos tienen partes comunes necesarias para el funcionamiento como, por ejemplo, acceso a base de datos, validación de formularios o seguridad. Un framework nos evita tener que programar estas partes, de esta manera nos resulta más fácil centrarnos en programar la aplicación.
- **Utilizar buenas prácticas.** Los frameworks están basados en patrones de desarrollo, normalmente MVC (Modelo-Vista-Controlador) que ayudan a separar los datos y la lógica de negocio de la interfaz con el usuario.
- **Desarrollar más rápido**. Desarrollar una aplicación con un framework nos permite hacerlo más rápido, más limpio y más seguro.

Como ejemplo en este curso vamos a instalar y utilizar Symfony, que es uno de los frameworks PHP más populares.

2. Symfony

2.1. Instalación y configuración

La instalación se puede hacer con Composer (que por cierto vamos a necesitar), pero el equipo de Symfony ha desarrollado un **instalador CLI** (Command Line Interface) propio que ofrece alguna funcionalidad adicional. Para descargar lo ejecutaremos el siguiente comando:

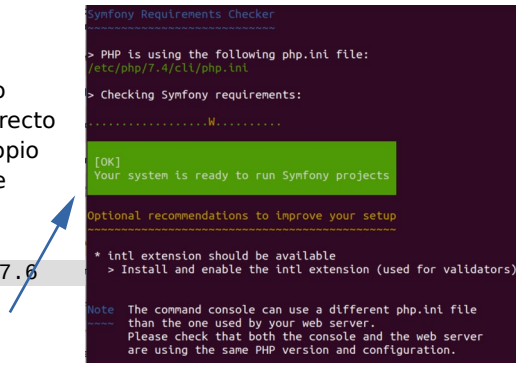
```
$ wget https://get.symfony.com/cli/installer -O - | bash
```

Cuando termine de descargar lo movemos a una carpeta local para instalarlo globalmente:

```
$ sudo mv /home/usuario/.symfony5/bin/symfony /usr/local/bin/symfony
```

Podemos comprobar que todo esté correcto con:

```
$ symfony check:requirements
```



Symfony usa un servidor web propio y por tanto no necesita que exista otro previo. En cambio, necesita algunos componentes ya instalados para el correcto funcionamiento. Estos componentes pueden ser muy variados, desde el propio intérprete PHP hasta extensiones del mismo como SimpleXML, en cuyo este último caso nos toca proceder así::

```
$ sudo apt-get install php7.6-xml   // U otra versión != 7.6
```

El resultado del chequeo interesa sin errores (marco en rojo) y podría ser el siguiente:

¡Atención!

Se recomienda que instalemos la extensión intl. En Ubuntu se puede hacer de la siguiente manera:

```
sudo apt install php-intl
```

Una vez tenemos instalado Symfony, nos situamos en donde queramos crear la carpeta de la aplicación web y ejecutamos:

```
symfony new mi-proyecto --webapp        // Opción --full en versiones anteriores Symfony
```

Esto crea un esqueleto de una **aplicación web Symfony completa** (es lo que indica **webapp**, también podemos desarrollar APIs o servicios que no necesiten tantos componentes) l**lamada mi-proyecto**, descargando Symfony y todas sus dependencias con Composer.

Si no tenemos configurado el usuario de git nos presentará un error y nos ofrece las posibilidades siguientes:

```
$ git config --global user.email "you@example.com"
$ git config --global user.name "Your Name"
```

Además, es el momento de apreciar si tenemos o no instalado **Composer**, en cuyo caso negativo se nos presenta el siguiente mensaje a la hora de crear un new proyecto Symfony.

```
* Creating a new Symfony project with Composer

  WARNING: Unable to find Composer, downloading one. It is recommended to install
      Composer yourself at https://getcomposer.org/download/

  (running /home/usuario/.symfony5/composer/composer.phar create-project
      symfony/skeleton /home/usuario/mi-proyecto  --no-interaction).
```

Error

Si tienes algún error durante la instalación, puedes probar a ejecutar:

```
composer clear-cache
```

Aunque para producción utilicemos servidores como Apache o Nginx, **Symfony integra un servidor web** muy cómodo a la hora de desarrollar. Vamos a lanzarlo y a acceder a la aplicación para comprobar que todo haya ido bien:

```
cd mi-proyecto
symfony server:start
```

Si queremos activar soporte local para SSL, debemos ejecutar:

```
symfony server:ca:install
```

Por defecto el servidor funciona en **http://localhost:8000/** . Si accedemos a esa URL podemos comprobar si todo ha ido bien. Nos aparecerá la interfaz de Symfony pero nos indicará, con error 404 en la esquina inferior izquierda, que nuestra aplicación está vacía. Es hora de crear una página.

Si has procedido a ejecutar todos los anteriores comandos puedes comprobar que los **requerimientos previos son importantes** y hay que cumplirlos para poder desarrollar con propiedad. También tienes que **ver y analizar la información facilitada** tanto al crear un nuevo proyecto como al desplegar el servidor web de Symfony.

2.2. Estructura de un proyecto Symfony

Los proyectos en Symfony tienen la siguiente estructura de carpetas:

- **config**: contiene la configuración de rutas, servicios y paquetes.
- **src:** código fuente (source code) PHP.
- **templates**: plantillas twig.

Estas son las carpetas más utilizadas, pero también hay otras:

- **bin**: archivos binarios (programas), incluyendo bin/console.
- **var**: ficheros que se crean automáticamente, como la caché o los logs.
- **vendor**: librerías externas (frameworks, extensiones...) descargadas con composer.
- **public**: directorio raíz del proyecto. Todo lo de esta carpeta es accesible al usuario.

2.3. Creando la primera página

Para crear una página hay que definir dos elementos:

- La **ruta**: la URL de la página. Indica qué controlador usar.
- El **controlador**: la función PHP que construye la página. Recibe la petición del usuario y crea un **objeto Response de Symfony,** que suele ser HTML pero también puede ser un archivo JSON, una imagen o incluso un PDF.

2.3.1. Ruta y controlador

***Aviso**: para permitir que vayas practicando con los ejemplos a medida que vas leyendo este documento vamos a diseñar, en esta primera página, un **único documento PHP controlador que tiene incluida tanto la lógica del negocio (controlador) como la presentación (vista)**, aunque esto no sea lo recomendado pero se hace por motivos didácticos, posibilitando la comprobación del resultado final de forma inmediata. En el apartado 2.3.2 vamos a separar esos dos componentes de MVC, la (V)ista y el (C)ontrolador, usando plantillas de presentación/vista.*

Vamos a crear una página **/sorteo/numero** que genere un número aleatorio (de ahí lo de sorteo) y nos lo muestre.

Para ello, crearemos una clase Controlador en **src/Controller/SorteoController.php** y le añadiremos un método que se encargue de hacerlo:

```
<?php
// src/Controller/SorteoController.php
namespace App\Controller;

use Symfony\Component\HttpFoundation\Response;

class SorteoController
{
    public function numero()
    {
        $numero = random_int(0, 100);

        return new Response(
            '<html><body>El resultado del sorteo es: '.$numero.'</body></html>'
        );
    }
}
```

Una vez creado el controlador, hay que definir una ruta que lo llame. Para ello editamos **config/routes.yaml** y lo dejamos de la siguiente manera:

```
# config/routes.yaml

app_numero_sorteo:
    path: /sorteo/numero
    controller: App\Controller\SorteoController::numero
```

Si tenemos el servidor activado, podemos probar a acceder a http://127.0.0.1:8000/sorteo/numero

2.3.2. Mostrando una plantilla

Hemos conseguido crear una URL que nos devuelve una respuesta (Response) en HTML, pero hemos integrado este HTML en el controlador, y ya hemos visto que tenemos que independizar todos los componentes.

Para dar entrada al componente Vista de MVC, Symfony incluye **Twig**, que es una herramienta para construir plantillas (normalmente en HTML) a las que le pasaremos los datos y que generarán la Response que le enviaremos al usuario.

Las plantillas se crean en el directorio **templates**. Crearemos una carpeta dentro llamada **sorteo** y dentro de esta crearemos un archivo llamado **numero.html.twig** con el siguiente contenido:

```
{# templates/sorteo/numero.html.twig #}
<html>
    <head><title>Número aleatorio</title></head>
    <body>
        <h1>El resultado del sorteo es: {{ numero }}</h1>
    </body>
</html>
```

En twig, {# y #} se usan para comentarios y con {{ y }} imprimimos valores de variables cuyos valores nos enviará normalmente el controlador.

Para utilizar esta plantilla, en lugar de la clase básica Controller, podemos hacer uso de la clase **AbstractController**, la cual nos proporciona métodos como $this->render, que nos permite usar plantillas.

```
return $this->render('sorteo/numero.html.twig', array('numero' => $numero));
```

Veremos más sobre esto en el siguiente apartado.

Si aplicamos el cambio propuesto el controlador por tanto quedaría así:

```
<?php
// src/Controller/SorteoController.php
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;

class SorteoController extends AbstractController
{
    public function numero()
    {
        $numero = random_int(0, 100);

        return $this->render('sorteo/numero.html.twig', ['numero' => $numero]);
    }
}
```

Con esto le estamos diciendo al controlador que utilice la plantilla sorteo/numero.html.twig y que para la variable en twig "numero" utilice el valor de $numero.

2.4. Rutas

Hoy en día todas las aplicaciones webs usan **URLs amigables**. En lugar de URLs tradicionales como index.php?article_id=57 usamos URLs como /read/intro-to-Symfony.

Con Symfony podemos configurar que rutas tiene nuestra aplicación web de varias maneras. Una de ellas es editando el fichero config/routes.yaml..

Como hemos visto, para crear una ruta indicamos un nombre, la ruta en sí (path) y qué controlador debe responder a la petición (controller).

```
# config/routes.yaml

app_numero_sorteo:
    path: /sorteo/numero
    controller: App\Controller\SorteoController::numero
```

En las URLs podemos definir comodines e incluso pasarle su valor al controlador. Por ejemplo, modifiquemos el método numero de SorteoController para añadir un máximo:

```
<?php
// src/Controller/SorteoController.php
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;

class SorteoController extends AbstractController
{
    public function numero($maximo)
    {
        $numero = random_int(0, $maximo);

        return $this->render('sorteo/numero.html.twig', ['numero' => $numero]);
    }
}
```

Y modifiquemos en routes.yml la ruta:

```
app_numero_sorteo:
    path: /sorteo/numero/{maximo}
    controller: App\Controller\SorteoController::numero
```

Comprobamos que si accedemos a http://localhost:8000/sorteo/numero/10 nos aparecen resultados entre el 0 y el 10.

Al haber hecho esa modificación ahora la URL http://localhost:8000/sorteo/numero no funciona. Para arreglarlo podemos configurar la ruta para que tenga valores por defecto:

```
app_numero_sorteo:
    path: /sorteo/numero/{maximo}
    controller: App\Controller\SorteoController::numero
    defaults:
        maximo: 100
```

Para saber más ...

*Otra forma de indicar las notas es a través de **anotaciones** en el controlador. En lugar de tener un archivo con las rutas, en cada método del controlador se indica las rutas a las que está asociado. Más información **aquí**.*

Ejercicio 5.1

Prueba a crear una ruta como el ejemplo: /suma/15/30 que muestre el resultado de la suma de los dos números. Ponle a ambos números el valor por defecto 2.

2.5. Controladores

Los **controladores** son funciones PHP que leen información de un **objeto Request** y devuelven una respuesta (**Response**), que normalmente suele ser HTML pero puede ser XML, JSON, una descarga, una redirección...

2.5.1. Obtener URLs

En nuestro código puede ser importante obtener la URL de alguna de nuestras páginas. Podríamos escribirla tal cual, pero si decidiéramos cambiarla por lo que sea, tendríamos que buscar todas las ocurrencias de la URL y corregirlas. En

lugar de eso, con Symfony podemos usar el **método generateURL**, al que tenemos que pasarle obligatoriamente el nombre de la ruta que usamos en **routes.yml**

```
$URL = $this->generateURL('app_numero_sorteo');
```

Si esa URL tiene algún parámetro, se lo podemos pasar en un array como argumento:

```
$URL = $this->generateURL('app_numero_sorteo', array('maximo' => 10));
```

2.5.2. Redirección

Podemos redireccionar a una URL de nuestra aplicación o externa.

- De nuestra aplicación, usando redirectToRoute:

```
return $this->redirectToRoute('app_numero_sorteo');
```

- También podemos incluir parámetros:

```
return $this->redirectToRoute('app_numero_sorteo', array('maximo' => 10));
```

- Cualquier URL, usando redirect:

```
return $this->redirect('http://symfony.com/doc');
```

2.5.3. Accediendo a datos de la petición (GET, POST, Cookies...)

Si añadimos al controlador una variable de tipo Request podemos acceder a datos de la request:

```
public function index(Request $request)
{
    // Obtiene las variables $_GET['page'] y $_POST['page'] respectivamente.
    $request->query->get('page');
    $request->request->get('page');

    // Obtiene variables SERVER
    $request->server->get('HTTP_HOST');

    // Obtiene ficheros subidos con formularios
    $request->files->get('foo');

    // Obtiene el valor de una COOKIE
    $request->cookies->get('PHPSESSID');

    // Obtiene valores de la petición HTTP
    $request->headers->get('host');
    $request->headers->get('content_type');
```

Ejercicio 5.2

Modifica **SorteoController::numero** para que, si no recibimos un número entero, nos redireccione a esa misma ruta pero con un máximo de 0.

2.6. Creando y usando plantillas Twig

Los controladores se encargan de recibir la petición y procesarla, y para mostrar la información al usuario generan la respuesta ayudándose de plantillas. De esta manera estamos separando la información y su procesamiento (**Modelo y Controlador**) de la forma en la que se le muestra al usuario (**Vista**).

2.6.1. Plantillas

Una plantilla es un fichero de texto que puede generar cualquier formato basado en texto (HTML, XML, CSV, LaTeX...). Symfony utiliza un sistema de plantillas llamado **Twig** que tiene una sintaxis propia, más sencilla y por tanto más amigable. Ejemplo:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Welcome to Symfony!</title>
    </head>
    <body>
        <h1>{{ page_title }}</h1>

        <ul id="navigation">
            {% for item in navigation %}
                <li><a href="{{ item.href }}">{{ item.caption }}</a></li>
            {% endfor %}
        </ul>
    </body>
</html>
```

Twig define tres tipos de **sintaxis**:

- {{ ... }} "Dice algo": imprime el valor de la expresión.
- {% ... %} "Hace algo": controla la lógica: ifs, bucles...
- {# ... #} "Comenta algo": es el equivalente de /*...*/ en PHP.

Twig también tiene **filtros** con los que podemos modificar el contenido antes de imprimirlo. Por ejemplo si añadimos | upper a una variable la transforma en mayúsculas antes de imprimir:

```
{{ title }} {#  Imprime la variable title normal #}
{{ title|upper }} {# Imprime la variable title en mayúsculas #}
```

Twig soporta **bucles y condicionales**:

```
{% if dia == "Martes" %}
<p>Hoy es martes</p>
{% endif %}

<ul>
{% for i in 1..10 %}
    <li>{{i}}</li>
{% endfor %}
</ul>

<ul>
{% for nombre in listaNombres %}
    <li>{{nombre}}</li>
{% endfor %}
</ul>
```

Twig incluye una larga lista de etiquetas, filtros y funciones disponibles por defecto. Se parece a PHP pero tiene algunas diferencias sutiles y agradables. El siguiente ejemplo utiliza una función cycle de Twig para alternar entre ‘par’ e ‘impar’ dentro del bucle.

```
{% for i in 1..10 %}
    <div class="{{ cycle(['par', 'impar'], i) }}">
        <!-- some HTML here -->
    </div>
{% endfor %}
```

El siguiente ejemplo muestra que los bucles for incluyen una clausula else opcional para el caso en el que no se haga ninguna iteración (por ejemplo porque queramos recorrer una lista que esté vacía):

```
{% for usuario in usuarios %}
    {{usuario.nombre}}
{% else %}
    <p>La lista de usuarios está vacía</p>
{% endfor %}
```

Para saber más...

Puedes averiguar más sobre twig en la documentacion oficial.

2.6.2. Herencias en plantillas

Muchas veces las páginas web tienen elementos comunes: la cabecera, el footer, la barra de menú... Twig nos permite definir plantillas base con bloques que puedan ser sobreescritos por otras plantillas, utilizando **{% block NOMBRE_DEL_BLOQUE %}.**

Primero creemos una clase base en la que definimos varios bloques:

```
{# templates/sorteo/estructuraBase.html.twig #}
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>{% block titulo %}aplicación de prueba{% endblock %}</title>
    </head>
    <body>
        <div id="sidebar">
            {% block sidebar %}
                <h2>Menu</h2>
                <ul>
                    <li><a href="/">Inicio</a></li>
                    <li><a href="/sorteo/numero">Número</a></li>
                </ul>
            {% endblock %}
        </div>

        <div id="contenido">
            {% block contenido %}{% endblock %}
        </div>
    </body>
</html>
```

¡Atención!

En la carpeta templates ya tenemos una plantilla base creada por Symfony: base.html.twig. Sin embargo, nosotros hemos creado la nuestra para nuestro proyecto.

Con **{% extends 'RUTA_A_PLANTILLA' %}** podemos crear una plantilla que se basa en otra (es decir, que hereda de otra). Los cambios a la plantilla principal los podremos hacer sobrescribiendo los bloques definidos y volviéndolos a definir.

Por ejemplo, vamos a modificar la plantilla numero.html.twig para que herede de la estructura base:y

```
{# templates/sorteo/numero.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}

{% block titulo %}Sorteo entre 0 y {{maximo}}{% endblock %}

{% block contenido %}
<h1>El resultado del sorteo es: {{ numero }}</h1>
{% endblock %}
```

El resultado de esta plantilla es el mismo que estructuraBase.html.twig, solo que hemos sobrescrito el contenido del título de la página y del div #contenido.

Hazlo tu también

Modifica la plantilla numero.html.twig tu también. Fíjate que la plantilla debe recibir numero y maximo.

Si quisiéramos no reemplazar, sino añadir contenido al bloque, utilizamos {{ parent() }} para incluir el contenido original

```
{% block sidebar %}
      <h3>Tabla de contenidos</h3>

      {# ... #}

      {{ parent() }}
{% endblock %}
```

Ejercicio 5.3

Crea en el controlador un método SorteoController::index que genere la página de bienvenida. Debe actuar en la ruta / y utilizar una plantilla basada en estructuraBase.html.twig.

Ejercicio 5.4

Crea en el controlador un método SorteoController::euromillones que genere de forma aleatoria un array con los números de una apuesta de euromillones y la muestre. Debe actuar en la ruta /sorteo/euromillones y utilizar una plantilla basada en estructuraBase.html.twig. Añade la entrada al menú.

Para pensar

En el ejercicio 5.4 tenemos que crear apuestas del euromillón que consisten en cinco números entre 1 y 50 y dos estrellas entre 1 y 12 que además no deben repetirse y quedan mejor si se ordenan.

Esto es lógica de negocio, ya que es algo que tiene que ver con el negocio que estamos tratando (apuestas) no con la web, y por tanto no debería encargarse el controlador sino el modelo.

Lo correcto sería crear una clase ApuestaEuromillones que se encargue de generar una apuesta correcta, de modo que el controlador sólo tenga que crear un objeto de esta clase y pasarlo a plantilla. Veremos más sobre esto en apartados venideros.

2.6.3. Incluyendo unas plantillas dentro de otras

No sólo podemos hacer llamadas a plantillas desde el controlador, también podemos hacerlas desde otra plantilla con {% include('RUTA_A_PLANTILLA', PARÁMETROS) %} . Por ejemplo, pongamos que tenemos una plantilla que, dado un artículo, muestra una pequeña descripción de este:

```
{# templates/articulo/articulo_detalles.html.twig #}
<h2>{{ articulo.titulo }}</h2>
<h3 class="autor">Por {{ articulo.autor }}</h3>

<p>
      {{ articulo.descripcion }}
</p>
```

Si estamos haciendo una página que muestra un listado de artículos, podríamos usar el siguiente código:

```
{# templates/articulo/list.html.twig #}
{% extends 'layout.html.twig' %}

{% block body %}
   <h1>Artículos recientes<h1>

      {% for articulo in articulos %}
            {{ include('articulo/articulo_detalles.html.twig', { 'articulo': articulo }) }}
      {% endfor %}
{% endblock %}
```

A include() podemos pasarle un listado de parámetros y los valores que deben tomar en la forma { 'NOMBRE1': VALOR1, 'NOMBRE2': VALOR2 }.

Ejercicio 5.5

Crea una plantilla llamada menu.html.twig que genere el contenido del bloque sidebar de estructuraBase.html.twig e incluya el enlace a /sorteo/euromillones. Necesitará una variable llamada seccionActual que contenga el nombre de la sección actual y con la que detectaremos cual es la página actual y a su entrada del menú le añadiremos la clase actual:

```
{# templates/sorteo/menu.html.twig #}
<h2>Menu</h2>
<ul>
    <li {% if seccionActual == "inicio" %}class="actual"{% endif %}>
        <a href="/">Inicio</a>
    </li>
    <li {% if seccionActual == "numero" %}class="actual"{% endif %}>
        <a href="/sorteo/numero">Número</a>
    </li>
    <!-- Más opciones -->
</ul>
```

En el resto de plantillas que hemos hecho basadas en esta plantilla principal, sobreescribe el bloque sidebar incluyendo la plantilla menu.html.twig indicando la sección pertinente. Por ejemplo en numero.html.twig sería:

```
{# templates/sorteo/numero.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block sidebar %}
{{ include('sorteo/menu.html.twig', { 'seccionActual': "numero" }) }}
{% endblock %}
{% block titulo %}Sorteo entre 0 y {{maximo}}{% endblock %}
{% block contenido %}
<h1>El resultado del sorteo es: {{ numero }}</h1>
{% endblock %}
```

Modifica también estructuraBase.html.twig para incluir una línea CSS que resalte las entradas marcadas con la clase "actual":

```
<head>
    <meta charset="UTF-8">
    <title>{% block titulo %}Aplicación de prueba{% endblock %}</title>
    <style> .actual { font-weight: bold; } </style>
</head>
```

2.6.4. Enlaces en las plantillas

De forma análoga a generateURL() en el controlador, en Twig tenemos **path(),** que recibe el nombre de la ruta y devuelve el enlace:

```
<a href="{{ path('articulos') }}">Artículos</a>
```

En caso de que la ruta tenga parámetros, se incluyen a continuación del nombre:

```
<a href="{{ path('articulo_mostrar', {'id': articulo.id}) }}">
```

Estas rutas son relativas. Si queremos una ruta absoluta debemos usar la función URL():

```
<a href="{{ URL('articulos') }}">Artículos</a>
```

Ejercicio 5.6

Sustituye los enlaces del menú elaborado anteriormente por enlaces generados con Twig. Comprueba que si modificas las rutas en el archivo routes.yaml, los enlaces se actualizan automáticamente.

2.6.5. Enlazando Assets

Un **asset** en Symfony es un **recurso**: una imagen, una hoja de estilo css, un script javaScript… . Como deben estar accesibles para todo el mundo, en Symfony se colocan en la carpeta **public**.

Cuando incluimos estos recursos, normalmente los enlazamos con una URL y si están en distintas carpetas tenemos que tenerlo en cuenta para la ruta.

Symfony incorpora una función llamada **assets()** que se encarga de generar la ruta automáticamente. Para usarla primero debemos instalarla:

```
composer require symfony/asset
```

Imaginemos que en public, que es la carpeta donde pongo los recursos accesibles por los usuarios, tengo una carpeta llamada css, y dentro de esta un archivo css llamado style.css (css/style.css). Si quisiera añadir esa hoja de estilo a mi aplicación web, debería añadir:

```
<link href="{{ asset('css/style.css') }}" rel="stylesheet" />
```

2.6.6. Incluyendo JavaScript y CSS en twig

El siguiente método nos ayuda a entender un poco mejor la filosofía de Twig. Normalmente en desarrollo web tenemos unas hojas de estilo principales o unos archivos JavaScript que usamos en todas las páginas.

Luego solemos tener casos en los que para unas páginas concretas necesitamos añadir un script o una hoja de estilos. Symfony propone esta aproximación:

```
{# templates/base.html.twig #}
<html>
    <head>
        {# ... #}

        {% block stylesheets %}
            <link href="{{ asset('css/main.css') }}" rel="stylesheet" />
        {% endblock %}
    </head>
    <body>
        {# ... #}

        {% block javascripts %}
            <script src="{{ asset('js/main.js') }}"></script>
        {% endblock %}
    </body>
</html>
```

En resumen, la idea es crear unos bloques a los que llama stylesheets y javascripts en la plantilla base y que contienen los scripts y las hojas de estilo comunes a todas las páginas de la web.

La página que quiera añadir un nuevo script u hoja de estilo simplemente tendría que sobreescribir el módulo, añadiendo el contenido nuevo y llamando a parent() para que se imprima también lo que había previamente:

```
{# templates/contact/contact.html.twig #}
{% extends 'base.html.twig' %}

{% block stylesheets %}
    {{ parent() }}

    <link href="{{ asset('css/contact.css') }}" rel="stylesheet" />
{% endblock %}

{# ... #}
```

Ejercicio 5.7

Añade un bloque de hojas de estilo css y otro de javascript a estructuraBase.html.twig. Crea un enlace a una hoja de estilo en la que, entre otras mejoras estéticas, recoja la decoración del elemento de la clase "actual" del menú que habíamos incluido directamente en el HTML.

2.7. Formularios

Hasta ahora hemos manejado el **Controlador** y la **Vista**. Toca darle cuerpo al **Modelo**. Empezaremos abordando los formularios.

Symfony incluye un componente "form" que hace más sencillo la generación y el procesamiento de estos. El primer paso es instalarlo:

```
composer require symfony/form
```

2.7.1. Creando una entidad

Supongamos que vamos a trabajar con apuestas genéricas en el proyecto de prueba que estamos haciendo y necesitamos un formulario con el que añadirlas. Lo primero es crear la **entidad** apuesta (src/Entity/Apuesta.php):

```php
<?php
// src/Entity/Apuesta.php
namespace App\Entity;

class Apuesta
{
	protected $texto;
	protected $fecha;

	public function getTexto()
	{
		return $this->texto;
	}

	public function setTexto($texto)
	{
		$this->texto = $texto;
	}

	public function getFecha()
	{
		return $this->fecha;
	}

	public function setFecha(\DateTime $fecha = null)
	{
		$this->fecha = $fecha;
	}
}
```

Este script contiene una clase PHP que de momento no tiene nada de Symfony, pero es donde empezamos a vislumbrar el modelo.

Estamos creando objetos que manejaremos mediante la lógica de negocio y por tanto modelamos su comportamiento.

2.7.2. Construyendo el formulario

Añadimos la ruta a routes.yaml:

```yaml
app_nueva_apuesta:
		path: /sorteo/apuestas/nueva
		controller: App\Controller\SorteoController::nuevaApuesta
```

Modificamos SorteoController.php y añadimos las siguientes líneas "use" al comienzo del fichero, al lado de las ya existentes:

```
use App\Entity\Apuesta;
use Symfony\Component\Form\Extension\Core\Type\TextType;
use Symfony\Component\Form\Extension\Core\Type\DateType;
use Symfony\Component\Form\Extension\Core\Type\SubmitType;
use Symfony\Component\HttpFoundation\Request;
```

Añadimos el siguiente método al controlador:

```
public function nuevaApuesta(Request $request)
    {
        $apuesta = new Apuesta();
        /* Descomenta las siguientes líneas para rellenar la apuesta
            con información de prueba en lugar de estar vacía */
        // $apuesta->setTexto('2 13 34 44 48');
        // $apuesta->setFecha(new \DateTime('tomorrow'));

        $form = $this->createFormBuilder($apuesta)
            ->add('texto', TextType::class)
            ->add('fecha', DateType::class)
            ->add('save', SubmitType::class,
                array('label' => 'Añadir Apuesta'))
            ->getForm();

        return $this->render('sorteo/nuevaApuesta.html.twig', array(
            'form' => $form->createView(),
        ));
    }
```

Para saber más ...

Para fijar de forma manual la fecha de la apuesta debemos crear un objeto de la clase DateTime. Podemos crearlo a partir de una fecha en formato yyyy-mm-dd hh-mi-ss o usando palabras como today, yesterday o now:

```
// Fecha en formato yyyy-mm-dd hh-mi-ss
$apuesta->setFecha(new \DateTime('2020-10-21 23:15:12'));
// Fecha y hora actual
$apuesta->setFecha(new \DateTime('now'));
// Si no ponemos argumento también coge la fecha y hora actual
$apuesta->setFecha(new \DateTime());
// Fecha de ayer
$apuesta->setFecha(new \DateTime('yesterday'));
// Fecha de mañana
$apuesta->setFecha(new \DateTime('tomorrow'));
```

Para saber más ...

Podemos escoger distintas clases para un mismo tipo de dato. Por ejemplo podemos usar la clase TextareaType para el campo de texto (habría que incluir la correspondiente línea use al comienzo del archivo):

```
// ...
    ->add('texto', TextareaType::class)
// ...
```

También podemos configurar algunas opciones de los campos. Por ejemplo para renderizar el campo fecha con un sólo cuadro de texto en lugar de 3 selects podemos usar la siguiente configuración:

```
// ...
        ->add('fecha', DateType::class, ['widget' => 'single_text'])
// ...
```

En este código podemos ver que el controlador puede crear un formulario a partir de un objeto de una clase, indicándole para cada atributo de la clase a qué tipo de campo se corresponde. Además, el formulario mostrará para cada campo los valores que contenga el objeto.

Hay más tipos de campo de formulario: TextareaType, EmailType, IntegerType, MoneyType... Aquí puedes encontrar una lista.

2.7.3. Mostrando el formulario

Añadimos a menu.html.twig la entrada correspondiente:

```
<li {% if seccionActual == "nuevaApuesta" %}class="actual"{% endif %}>
        <a href="{{ path('app_nueva_apuesta') }}">Nueva Apuesta</a>
</li>
```

Y creamos la plantilla nuevaApuesta.html.twig que mostrará el formulario:

```
{# templates/sorteo/nuevaApuesta.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}

{% block titulo %}Nueva apuesta{% endblock %}

{% block contenido %}
{{ form_start(form) }}
{{ form_widget(form) }}
{{ form_end(form) }}
{% endblock %}

{% block sidebar %}
   {{ include('sorteo/menu.html.twig', { 'seccionActual': "nuevaApuesta" }) }}
{% endblock %}
```

¡Eso es todo! Sólo necesitamos tres líneas:

- form_start(form): inserta las etiquetas de apertura del formulario.
- form_widget(form): inserta los campos del formulario, incluyendo etiquetas y mensajes de error de validación.
- Form_end(form): inserta las etiquetas de cierre del formulario.

¡Ahora tú!

Sigue los pasos descritos en esta sección, accede a /sorteo/apuestas/nueva y comprueba que el formulario aparece. Descomenta las líneas que rellenan los datos del objeto $apuesta y comprueba que el formulario muestra la información correspondiente.

2.7.4. Gestionando el formulario

Por defecto, el formulario generado funciona con POST y lleva al mismo controlador que lo creó.

Para recoger la información del formulario, creamos el formulario y lo enlazamos con el objeto Request, que representa la petición del servidor y contiene los valores de las variables. Al hacer esto podem**os obtener un objeto con los valores** que han introducido en el formulario.

De momento no lo vamos a implementar, ya que no tenemos nada que hacer con el formulario hasta que veamos el siguiente apartado, pero sería así:

```
public function nuevaApuesta(Request $request)
{
    $apuesta = new Apuesta();
    $form = $this->createFormBuilder($apuesta)
        ->add('texto', TextType::class)
        ->add('fecha', DateType::class)
        ->add('save', SubmitType::class,
                array('label' => 'Añadir Apuesta'))
        ->getForm();

    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        // De esta manera podemos rellenar la variable $apuesta
        // con los datos del formulario.
        $apuesta = $form->getData();
        // Ahora podríamos guardarla en la base de datos, redirigir
        // a una página de confirmación, crear un mensaje...
    }

    return $this->render('sorteo/nuevaApuesta.html.twig', array(
        'form' => $form->createView(),
    ));
}
```

2.8. Base de datos y Doctrine

Symfony no tiene un componente que trabaje con bases de datos, sino que está integrado con **Doctrine**.

Doctrine es un ORM, un mapeador objeto-relacional como los que iste en Programación de 1º. Esto quiere decir que se encarga de "mapear" objetos en una base de datos, o lo que es lo mismo, almacenar y extraer objetos de una base de datos, modificarlos y borrarlos. El almacenar los objetos de un programa en algún sitio (base de datos, ficheros...) para poder recuperarlos en otro momento se denomina "persistencia".

2.8.1. Instalando doctrine

Para instalar doctrine lo hacemos a través de composer:

```
composer require symfony/orm-pack

composer require symfony/maker-bundle --dev
```

2.8.2. Configurando la base de datos

En la carpeta del proyecto Symfony podemos encontrar un archivo .env. Este archivo recoge la configuración de entorno de la aplicación (configuración del servidor de correo, de la base de datos...).

Como una aplicación puede estar siendo desarrollada por varias personas a la vez y cada uno tiene sus usuarios, sus rutas, sus contraseñas... usar todo el mundo el fichero .env no es buena idea, ya que si alguien sube un cambio al repositorio, cambia la configuración del resto. Para evitarlo podemos crear un **archivo .env.local** en el cual añadir nuestra configuración personal. Este archivo no se sube al repositorio (de hecho el .gitignore de Symfony lo bloquea por defecto) y Symfony dará prioridad a la configuración de este fichero frente a lo que ponga en .env.

En este fichero debemos añadir una línea que indique a Symfony cómo conectar con la base de datos, especificando el usuario, la clave, el servidor, el puerto y el nombre de la base de datos:

```
DATABASE_URL="mysql://USUARIO:CLAVE@SERVIDOR:PUERTO/NOMBREBBDD"
```

Un ejemplo podría ser:

```
DATABASE_URL="mysql://dwes:abc123@127.0.0.1:3306/symfonyBlog"
```

Se pueden usar otros SGBD, no sólo MySQL. Un ejemplo para usar SQLite sería:

```
DATABASE_URL="sqlite:///%kernel.project_dir%/var/app.db"
```

Ahora tú

Crea una base de datos en MySQL para estos ejemplos. Crea el fichero .env.local en la carpeta de Symfony y añade la línea de configuración de la base de datos.

2.8.3. Creando la entidad

En el apartado anterior hemos visto que creamos entidades definiendo clases en la carpeta **Entity**. Para que Doctrine reconozca estas entidades y las almacene en la base de datos, le debemos dar información adicional: tipo de dato, si puede ser nulo, longitud…

Todos estos parámetros se indican como comentarios (anotaciones) en la clase. Podemos utilizar un asistente de creación de entidades de Symfony que, mediante preguntas, crea el código de la clase con las anotaciones incluidas. Para ello se ejecuta en la carpeta del proyecto Symfony el siguiente comando:

```
php bin/console make:entity
```

Por ejemplo, si borramos la entidad Apuesta.php y la volviéramos a crear con este comando (texto como string y fecha como datetime, ambas con la longitud y la nulidad por defecto), obtendríamos lo siguiente:

```
<?php
namespace App\Entity;
use Doctrine\ORM\Mapping as ORM;
/**
 * @ORM\Entity(repositoryClass="App\Repository\ApuestaRepository")
 */
class Apuesta
{
    /**
     * @ORM\Id()
     * @ORM\GeneratedValue()
     * @ORM\Column(type="integer")
     */
    private $id;
    /**
     * @ORM\Column(type="string", length=255)
     */
    private $texto;
    /**
     * @ORM\Column(type="datetime")
     */
    private $fecha;

    public function getId(): ?int
    {
        return $this->id;
    }
    public function getTexto(): ?string
    {
        return $this->texto;
    }
```

```
    public function setTexto(string $texto): self
    {
        $this->texto = $texto;
        return $this;
    }
    public function getFecha(): ?\DateTimeInterface
    {
        return $this->fecha;
    }
    public function setFecha(\DateTimeInterface $fecha): self
    {
        $this->fecha = $fecha;
        return $this;
    }
}
```

Este método genera automáticamente un campo ID, por lo que no tenemos que incluirlo. También genera automáticamente una clase ApuestaRepository en /src/Repository/ApuestaRepository.php que se encarga de generar las consultas correspondientes para obtener Apuestas de la base de datos. Trabajaremos con repositorios más adelante.

Ahora tú

Borra la entidad Apuesta.php y créala usando el comando ***php bin/console make:entity*** como indica el ejemplo. Comprueba que la entidad se ha creado correctamente y el formulario sigue funcionando.

Es posible que al borrar e intentar crear de nuevo la clase de un error porque detecte que antes existía y trate de modificarla en lugar de crearla:

```
In DebugClassLoader.php line 345:

Warning: include(/home/usuario/mi-proyecto/vendor/composer/../../src/Entity/
Apuesta.php): failed to open stream: No such file or directory
```

En ese caso ejecutamos la siguiente línea para regenerar la carga automática de clases:

```
composer dump-autoload
```

Una vez creada la entidad, podemos hacer que **Doctrine cree la tabla correspondiente**. Para ello ejecutamos el comando que genera las migraciones, es decir, un archivo PHP con las consultas SQL necesarias para actualizar la base de datos al estado actual:

```
php bin/console make:migration
```

Si todo ha ido bien, ejecutamos el comando que hace que Doctrine ejecute las migraciones pendientes:

```
php bin/console doctrine:migrations:migrate
```

Ahora tú

Ejecuta los comandos ***php bin/console make:migration*** y p***hp bin/console doctrine:migrations:migrate*** y comprueba que las tablas se han creado en tu base de datos. Echa un vistazo al archivo creado en la carpeta ***migrations***.

2.8.4. Persistiendo los objetos en la base de datos

Para almacenar los objetos en la base de datos, tan sólo hemos de indicar a Doctrine qué objeto queremos almacenar e indicarle cuándo hacer los cambios.

Vamos a modificar el método del controlador que gestionaba el formulario de añadir apuestas:

```
public function nuevaApuesta(Request $request)
{
	$apuesta = new Apuesta();
	// Rellenamos el objeto con información de prueba
	// $apuesta->setTexto('2 13 34 44 48');
	// $apuesta->setFecha(new \DateTime('tomorrow'));
	$form = $this->createFormBuilder($apuesta)
		->add('texto', TextType::class)
		->add('fecha', DateType::class)
		->add('save', SubmitType::class,
			array('label' => 'Añadir Apuesta'))
		->getForm();

	$form->handleRequest($request);

	if ($form->isSubmitted() && $form->isValid()) {
		// De esta manera podemos rellenar la variable
		// $apuesta con los datos del formulario.
		$apuesta = $form->getData();

		// Obtenemos el gestor de entidades de Doctrine
		$entityManager = $this->getDoctrine()->getManager();

		// Le decimos a doctrine que nos gustaría almacenar
		// el objeto de la variable en la base de datos
		$entityManager->persist($apuesta);

		// Ejecuta las consultas necesarias
		$entityManager->flush();

		//Redirigimos a una página de confirmación.
		return $this->redirectToRoute('app_apuesta_creada');
	}

	return $this->render('sorteo/nuevaApuesta.html.twig', array(
		'form' => $form->createView(),
	));
}
```

Además vamos a añadir una página de confirmación, añadiendo un nuevo método al controlador:

```
public function apuestaCreada()
{
	return $this->render('sorteo/apuestaCreada.html.twig');
}
```

Una nueva plantilla:

```
{# templates/sorteo/apuestaCreada.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block contenido %}
<strong>Su petición ha sido efectuada correctamente</strong>
{% endblock %}
{% block sidebar %}
   {{ include('sorteo/menu.html.twig', { 'seccionActual': "nuevaApuesta" }) }}
{% endblock %}
```

Y una nueva ruta:

```
app_apuesta_creada:
        path: /apuesta/creada
        controller: App\Controller\SorteoController::apuestaCreada
```

Prueba a acceder de nuevo a la página de añadir apuestas y añade una. Comprueba en la base de datos que la apuesta se crea.

2.8.5. Obteniendo objetos de la base de datos

Doctrine permite obtener la información de la base de datos de distintas maneras. Tiene un lenguaje propio (**DQL, Doctrine Query Language**) y permite hacer también consultas SQL directamente, pero también crea unas clases especiales llamadas "**Repositorios**" que no necesitan de ningún lenguaje.

Vamos a añadir una página que nos permita consultar los datos de una apuesta. Modificamos SorteoController.php y añadimos el siguiente método:

```
    public function verApuesta($id)
    {
        // Obtenemos el gestor de entidades de Doctrine
        $entityManager = $this->getDoctrine()->getManager();

        /* Obtenemos el repositorio de Apuestas y  buscamos en el usando la id de la apuesta */
        $apuesta= $entityManager->getRepository(Apuesta::class)->find($id);

        // Si la apuesta no existe lanzamos una excepción.
        if (!$apuesta){
            throw $this->createNotFoundException(
                'No existe ninguna apuesta con id '.$id
            );
        }

        /* Pasamos la apuesta a una plantilla que  se encargue de mostrar sus datos. */
        return $this->render('sorteo/verApuesta.html.twig', array(
            'apuesta' => $apuesta,
        ));
    }
```

Creamos la plantilla verApuesta.html.twig para ver los datos de la apuesta:

```
{# templates/sorteo/verApuesta.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block contenido %}
<h1>Apuesta #{{apuesta.id}}</h1>
<p>Apuesta: {{ apuesta.texto }} ({{ apuesta.fecha|date("d/m/Y") }})</p>
{% endblock %}
{% block titulo %}Apuesta {{apuesta.id}}{% endblock %}
```

Y añadimos la ruta

```
app_apuesta_ver:
        path: /apuesta/ver/{id}
        controller: App\Controller\SorteoController::verApuesta
```

Si utilizamos **find**, Symfony espera que le pasemos la ID del objeto a buscar. Tenemos varias opciones más. Imaginemos que tuviéramos una clase Producto, tendríamos opciones como estas:

```
// Almaceno en una variable el repositorio
$repository = $this->getDoctrine()->getRepository(Product::class);

// Buscar por clave primaria (normalmente 'id')
$product = $repository->find($id);

// Buscar un solo elemento por su atributo name
$product = $repository->findOneBy(['name' => 'Keyboard']);

// O buscar uno por name y price
$product = $repository->findOneBy([
        'name' => 'Keyboard',
        'price' => 1999,
]);

// Buscar todos los productos con ese valor de name, ordenados por precio
$products = $repository->findBy(
        ['name' => 'Keyboard'],
        ['price' => 'ASC']
);

// Obtener todos los objetos de tipo
$products = $repository→findAll();
```

¡Ahora tú!

Prueba a ver una de las apuestas que hayas creado.

Para terminar, vamos a crear la **página principal de la sección apuestas,** donde listemos todas las apuestas y ofrezcamos la posibilidad de añadir una nueva apuesta. Añadimos el método al controlador:

```
    public function listaApuestas()
      {
            // Obtenemos el gestor de entidades de Doctrine
            $entityManager = $this->getDoctrine()->getManager();

            // obtenemos todas las apuestas
            $apuestas= $entityManager->getRepository(Apuesta::class)->findAll();

            return $this->render('sorteo/listaApuestas.html.twig', array(
                  'apuestas' => $apuestas,
            ));
      }
```

Añadimos el enlace a la lista al menú en menu.html.twig:

```
{* ... *}
<li {% if seccionActual == "apuestas" %}class="actual"{% endif %}>
      <a href="{{ path('app_apuesta_lista') }}">Apuestas</a>
</li>
{* ... *}
```

Añadimos el bloque del menú a verApuesta.html.twig:

```
{# templates/sorteo/verApuesta.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block contenido %}
<h1>Apuesta #{{apuesta.id}}</h1>
Apuesta: {{ apuesta.texto }} ({{ apuesta.fecha|date("d/m/Y") }})
{% endblock %}
{% block titulo %}Apuesta {{apuesta.id}}{% endblock %}
{% block sidebar %}
   {{ include('sorteo/menu.html.twig', { 'seccionActual': "apuestas" }) }}
{% endblock %}
```

Creamos la correspondiente plantilla listaApuestas.html.twig:

```
{# templates/sorteo/listaApuestas.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block sidebar %}
   {{ include('sorteo/menu.html.twig', { 'seccionActual': "apuestas" }) }}
{% endblock %}
{% block contenido %}
      <h1>Apuestas</h1>
      <ul>
            {% for apuesta in apuestas %}
                  <li><a href="{{ path('app_apuesta_ver', {'id': apuesta.id}) }}">
                        Apuesta #{{ apuesta.id }}
                  </a></li>
            {% else %}
                  <li>No hay apuestas</li>
            {% endfor %}

      </ul>
      <p><a href="{{ path('app_nueva_apuesta') }}">Añadir apuesta</a></p>
{% endblock %}
```

Y añadimos la ruta:

```
app_apuesta_lista:
      path: /apuesta/
      controller: App\Controller\SorteoController::listaApuestas
```

2.8.6. Editando objetos

Para editar objetos, los obtenemos de la base de datos y los modificamos. No es necesario hacer persist con el entity manager, ya que los objetos ya estaban en la base de datos y Doctrine asume que si hacemos cambios en ellos, queremos que se guarden.

Vamos a añadir una página para editar las tareas. Añadimos el siguiente método al controlador:

```
public function editarApuesta(Request $request, $id)
{
      // Obtenemos el gestor de entidades de Doctrine
      $entityManager = $this->getDoctrine()->getManager();

      // Obtenemos el repositorio de Apuestas y buscamos en el usando la id de la apuesta
      $apuesta = $entityManager->getRepository(Apuesta::class)->find($id);
      // Si la apuesta no existe lanzamos una excepción.
      if (!$apuesta){
            throw $this->createNotFoundException(
                  'No existe ninguna apuesta con id '.$id
            );
      }

      // Creamos el formulario a partir de $apuesta
      $form = $this->createFormBuilder($apuesta)
            ->add('texto', TextType::class)
            ->add('fecha', DateType::class)
            ->add('save', SubmitType::class, array('label' => 'Editar Apuesta'))
            ->getForm();

      $form->handleRequest($request);

      if ($form->isSubmitted() && $form->isValid()) {
            // De esta manera podemos sobrescribir la variable $apuesta con los datos del formulario.
```

```
            $apuesta = $form->getData();

            // Ejecuta las consultas necesarias (UPDATE en este caso)
            $entityManager->flush();

            //Redirigimos a la página de ver la apuesta editada.
            return $this->redirectToRoute('app_apuesta_ver', array('id'=>$id));
        }

        return $this->render('sorteo/nuevaApuesta.html.twig', array(
            'form' => $form->createView(),
        ));
}
```

Para editar la apuesta no vamos a crear una nueva plantilla, sino que vamos a reutilizar el formulario de añadir apuesta. Una vez editamos la apuesta, mandamos al usuario a la página donde puede ver la apuesta editada.

Quedaría añadir la siguiente ruta:

```
app_apuesta_editar:
    path: /apuesta/editar/{id}
    controller: App\Controller\SorteoController::editarApuesta
```

Podemos también modificar templates/sorteo/verApuesta.html.twig y añadir un enlace para editar la apuesta:

```
{# templates/sorteo/verApuesta.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block contenido %}
<h1>Apuesta #{{apuesta.id}}</h1>
<p>Apuesta: {{ apuesta.texto }} ({{ apuesta.fecha|date("d/m/Y") }})</p>
<p><a href="{{ path('app_apuesta_editar', {'id' : apuesta.id}) }}">Editar</a></p>
{% endblock %}
{% block titulo %}Apuesta {{apuesta.id}}{% endblock %}
{% block sidebar %}
   {{ include('sorteo/menu.html.twig', { 'seccionActual': "apuestas" }) }}
{% endblock %}
```

2.8.7. Borrando objetos

Para borrar objetos debemos obtenerlos de la base de datos e indicar que los queremos borrar con el método remove() del entity manager.

Añadimos el siguiente método al controlador:

```
public function borrarApuesta($id)
    {
        // Obtenemos el gestor de entidades de Doctrine
        $entityManager = $this->getDoctrine()->getManager();

        // Obtenemos repositorio de Apuestas y buscamos en el usando la id de la apuesta
        $apuesta= $entityManager->getRepository(Apuesta::class)->find($id);

        // Si la apuesta no existe lanzamos una excepción.
        if (!$apuesta){
            throw $this->createNotFoundException(
                'No existe ninguna apuesta con id '.$id
            );
        }
        $entityManager->remove($apuesta);
        $entityManager->flush();
```

```
        return $this->render('sorteo/apuestaBorrada.html.twig');
    }
```

La plantilla apuestaBorrada.html.twig sería la siguiente:

```
{# templates/sorteo/apuestaBorrada.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block contenido %}
<strong>Su apuesta ha sido borrada correctamente</strong>
{% endblock %}
{% block titulo %}Borrar apuesta{% endblock %}
{% block sidebar %}
  {{ include('sorteo/menu.html.twig', { 'seccionActual': "apuestas" }) }}
{% endblock %}
```

Y quedaría añadir la ruta:

```
app_apuesta_borrar:
    path: /apuesta/borrar/{id}
    controller: App\Controller\SorteoController::borrarApuesta
```

Podemos también modificar templates/sorteo/verApuesta.html.twig y añadir un enlace para borrar la apuesta:

```
{# templates/sorteo/verApuesta.html.twig #}
{% extends 'sorteo/estructuraBase.html.twig' %}
{% block contenido %}
<h1>Apuesta #{{apuesta.id}}</h1>
<p>Apuesta: {{ apuesta.texto }} ({{ apuesta.fecha|date("d/m/Y") }})</p>
<p><a href="{{ path('app_apuesta_editar', {'id' : apuesta.id}) }}">Editar</a></p>
<p><a href="{{ path('app_apuesta_borrar', {'id' : apuesta.id}) }}">Borrar</a></p>
{% endblock %}
{% block titulo %}Apuesta {{apuesta.id}}{% endblock %}
{% block sidebar %}
  {{ include('sorteo/menu.html.twig', { 'seccionActual': "apuestas" }) }}
{% endblock %}
```